WEB PROGRAMLAMA I · SCÜ ŞARKIŞLA UBYO

JavaScript'in Kalbi: Event Loop ve Asenkron Mimari

Sunan: Ayşe Nurcihan Akabaş
Grup: Altay

GİRİŞ

JavaScript nedir?

Web'in programlama dili

HTML iskelet, CSS boya, JavaScript elektrik: sayfayı canlı ve etkileşimli yapar.

Her yerde çalışır

Tarayıcıda, sunucuda (Node.js), mobil ve masaüstü uygulamalarda.

Çekirdek dil + API'ler

Dilin kendisi küçük; DOM, fetch, zamanlayıcılar tarayıcının sunduğu API'lerdir.

```
console.log("Merhaba, dünya!"); // ilk satırın
```

Benzetme HTML binanın iskeleti, CSS boyası, JavaScript elektriği: elektrik olmazsa bina durur ama ışık yanmaz.

GİRİŞ

JavaScript'in kısa tarihi

1995

Brendan Eich, Netscape'te ilk sürümü yaklaşık 10 günde yazar. Adı Mocha, sonra LiveScript, sonunda JavaScript olur.

1997

ECMAScript: dil standartlaşır. Tarayıcılar aynı kuralları izler.

2009

Node.js: JavaScript sunucuya taşınır, tek dille hem ön hem arka yüz yazılır.

2015

ES6: let, const, ok fonksiyon, class ve Promise gelir.

Bugün

async/await standart; her yıl yeni ECMAScript sürümü yayımlanır.

Benzetme JavaScript ile Java'nın akrabalığı yok: yalnızca pazarlama adı. Halıyla halıfleks gibi.

GİRİŞ

JavaScript'in özellikleri

Yorumlanan / JIT

Motor (V8 gibi) kodu çalışırken derler.

Dinamik tipli

Değişkenin tipi çalışırken belli olur.

Tek iş parçacıklı

Aynı anda tek bir kod satırı çalışır.

Çok paradigmalı

Nesne yönelimli, fonksiyonel ve olay güdümlü yazılabilir.

Birinci sınıf fonksiyonlar

Fonksiyon değişkene atanır, parametre olarak geçer.

Olay güdümlü

Tıklama, zamanlayıcı, ağ cevabı gibi olaylara tepki verir.

Benzetme Tek iş parçacığı, tek kasalı bir market gibi: müşteriler sırayla, tek tek hizmet alır.

Değişkenler, veri tipleri ve operatörler

```
const ad = "name"; // string, değişmez
let yas = 20; // number, değişebilir
let aktif = true; // boolean
yas += 1; // 21
yas > 18 && aktif; // true
typeof ad; // "string"
```

Veri tipleri

string, number, boolean, null, undefined, object

Operatörler

+ - * / % matematik. === !== > < karşılaştırma. && || ! mantık.

Benzetme Değişken, üstü etiketli bir kutu gibi: içine değer koyarsın, etiketiyle çağırırsın.

JS TEMELLERİ

Koşullar ve döngüler

```
const puan = 70;
if (puan ⩾ 50) {
  console.log("Geçti");
} else {
  console.log("Kaldı");
}
```

```
for (let i = 1; i ⩽ 3; i++) {
  console.log(i); // 1, 2, 3
}
let n = 3;
while (n > 0) { n--; }
```

Benzetme if yol ayrımındaki tabela gibi karar verir; döngü stadyumda tur atmak gibi aynı işi tekrarlar.

JS TEMELLERİ

Fonksiyonlar ve dizi metotları

```
function selamla(ad) {
  return "Merhaba " + ad;
}
const topla = (a, b) ⇒ a + b;
selamla("name"); // "Merhaba name"
```

```
const s = [1, 2, 3, 4];
s.push(5);
s.map(x ⇒ x * 2); // [2,4,6,8,10]
s.filter(x ⇒ x > 2); // [3,4,5]
```

Benzetme Fonksiyon yemek tarifi gibi: bir kez yaz, istediğin kadar pişir. Callback de bir fonksiyondur.

JS TEMELLERİ

Nesneler, DOM ve olaylar

```
const kisi = { ad: "name", yas: 20 };
kisi.ad; // "name"
const j = JSON.stringify(kisi);
JSON.parse(j).yas; // 20
```

```
const btn = document.querySelector("button");
btn.addEventListener("click", () ⇒ {
  console.log("Tıklandı!");
});
```

Kabarcıklanma ve Event Loop

Olay önce tıklanan öğede, sonra üst öğelere doğru yayılır. Tıklama callback'i task kuyruğuna girer: Event Loop'a köprü bu.

Benzetme Olay dinleyici kapı zili gibi: biri basınca (click) bekleyen kod çalışır.

EVENT LOOP

Event Loop nedir?

Call Stack

Şu an çalışan fonksiyonların yığını.

Web API'ler

Bekleyen işleri (zamanlayıcı, ağ) arka planda yürütür.

Kuyruklar

Biten işlerin geri dönmek için sıra beklediği yer.

Event Loop

Stack boşsa kuyruktan bir iş alıp stack'e koyan döngü.

Benzetme: Tek garsonlu restoran. Siparişi alır, mutfağa verir, beklemeden diğer masaya geçer; yemek hazır olunca servis eder.

EVENT LOOP

Web API'ler ve kuyruklar

Web API'ler

setTimeout, fetch, DOM olayları JS motorunun değil tarayıcının parçasıdır. İşi arka planda yürütür.

Task kuyruğu

Biten zamanlayıcılar ve olay geri çağrıları burada sıra bekler.

Microtask kuyruğu

Promise.then ve await devamları burada bekler. Önceliklidir.

```
console.log("istek gidiyor");
fetch("/veri").then(r ⇒ r.json()).then(v ⇒ console.log(v));
console.log("beklemeden devam"); // cevap en son gelir
```

Benzetme Kargo gibi: siparişi verirsin (fetch), kargo yolda (Web API), kapına gelince zil çalar (kuyruk + event loop).

ASENKRON MİMARİ

Asenkron mimari nedir?

Senkron

İşler sırayla yürür; biri bitmeden diğeri başlamaz. Uzun iş sayfayı dondurur.

Asenkron

Uzun iş başlatılır, beklenmez. Bitince sonucu bir geri çağrı (callback) alır.

```
console.log("Başla");
setTimeout(() ⇒ console.log("2 sn sonra"), 2000);
console.log("Bitir"); // sırayla: Başla, Bitir, 2 sn sonra
```

Benzetme Çamaşır makinesini çalıştırıp başında beklemeden yemek yapmak gibi: iş arka planda biter, sen yoluna devam edersin.

İLİŞKİ

Üçü birlikte nasıl çalışır?

1- Kod
Satırlar stack'e girer

2- Web API
setTimeout, fetch devreder

3- Kuyruk
Biten iş sıraya girer

4- Event Loop
Stack boşsa bir iş taşır

5- Stack
Callback çalışır

Senkron kod

Her zaman önce stack'te biter.

Asenkron kod

Önce stack boşalır, sonra sırası gelir.

Event Loop

İkisini birbirine bağlayan köprüdür.

Benzetme Fabrika bandı gibi: malzeme (kod) gelir, makine (Web API) işler, bant (kuyruk) taşır, işçi (event loop) teslim alır.

ÖRNEK

Çıktı ne olur?

```
console.log("1");
setTimeout(() ⇒ console.log("2"), 0);
Promise.resolve()
  .then(() ⇒ console.log("3"));
console.log("4");
```

Cevap: 1, 4, 3, 2

1 ve 4 senkron, hemen yazılır.
3 microtask, 2'den önce gelir.
2 task kuyruğunda, en son çalışır.

Sıfır saniye "hemen" demek değildir: sadece kuyruğa girer.

ÖRNEK

Örneği adım adım izleyelim

1- log("1")

Stack'te çalışır, ekrana 1 yazılır.

2- setTimeout

Web API'ye devredilir; callback task kuyruğuna gider.

3- Promise.then

Callback microtask kuyruğuna girer.

4- log("4")

Ekrana 4 yazılır. Stack boşalır.

5- Event Loop

Önce microtask: 3. Sonra task: 2. Sonuç: 1, 4, 3, 2.

Benzetme Sınavda kolay soruları önce bitirip zor soruyu sona bırakmak gibi: senkron önce, asenkron sonra.

CALL STACK

Call Stack nasıl işler?

```
function c() { console.log("c"); }
function b() { c(); }
function a() { b(); }
a();
```

Yığın (LIFO)

a girer, b girer, c girer.
En son giren en önce çıkar: c, b, a.

Taşma (stack overflow)

Sonsuz özyineleme yığını doldurur ve hata verir.

Benzetme Tabak yığını gibi: en son koyduğun tabağı ilk alırsın.

KARŞILAŞTIRMA

Task ve Microtask

Task (Macrotask)

setTimeout, setInterval, tıklama olayları, I/O.
Her turda yalnızca bir tane alınır.

Microtask

Promise.then, queueMicrotask, await sonrası.
Stack boşalınca hepsi bitene kadar çalışır.

Sıra: senkron kod, tüm microtask'lar, (ekran çizimi), sıradaki tek task.

Benzetme Banka gibi: VIP müşteri (microtask) sıradakilerden (task) önce hep hizmet alır.

KARŞILAŞTIRMA

Callback, Promise, async/await

Callback

```
fs.readFile(f, (hata, v)
⇒ {
  console.log(v);
});
```

Promise

```
fetch(url)
 .then(r ⇒ r.json())
 .catch(h ⇒
console.log(h));
```

async / await

```
async function al() {
  const r = await
fetch(url);
  const v = await
r.json();
}
```

Aynı iş, üç yazım: callback iç içe kaçar, Promise zincirlenir, async/await senkron gibi okunur.

ÖZEL KARAKTERLER

JavaScript'in özel karakterleri

⇒

Ok fonksiyon: fonksiyonu kısa yazmanın yolu.

İşe yarar: Callback'lerde kod kısalır, okunması kolaylaşır.

```
const kare = x ⇒ x * x;
```

${ }

Şablon metin: `` (ters tırnak) içine değişken gömer.

İşe yarar: "+" ile metin birleştirme derdi biter.

```
`Merhaba ${ad}, ${yas} yaşında`
```

• • •

Yayma / toplama: diziyi veya nesneyi açar ya da toplar.

İşe yarar: Kopyalamak ve birleştirmek çok kolaylaşır.

```
const yeni = [...dizi, 4];
```

?.

Opsiyonel zincir: yoksa hata vermez, undefined döner.

İşe yarar: API'den boş veri gelince programın çökmesini önler.

```
kisi?.adres?.sehir
```

??

Null birleştirme: yalnızca null veya undefined ise varsayılanı verir.

İşe yarar: 0 ve "" geçerli değerdir; || bunları siler, ?? silmez.

```
const adet = x ?? 0;
```

≡

Sıkı eşitlik: değer ve tipi birlikte karşılaştırır.

İşe yarar: == tipi çevirir ve şaşırtır: "5" == 5 true çıkar.

```
"5" ≡ 5 // false
```

Benzetme Kısaltmalar telgraf gibi: az karakterle çok şey anlatır.

YAZIM KURALLARI

Kod yazımının özellikleri

Alışkanlıklar

- Adlar camelCase: kullaniciAdi
- Önce const, değişecekse let
- Büyük/küçük harf duyarlıdır
- Noktalı virgül önerilir
- Bloklar süslü parantezle

```
"use strict";
const kullaniciAdi = "name";
let sayac = 0;
// Yorum satırı
/* Çok satırlı yorum */
```

Benzetme Dil bilgisi kuralları gibi: uymazsan okuyan (motor) seni anlayamaz.

HATALAR

Sık yapılan hatalar

== yerine ===

"5" == 5 true, "5" === 5 false.

var kullanmak

Kapsam karışır; let ve const kullan.

await unutmak

Promise beklenmez, sonuç Promise gelir.

Sonsuz döngü

while(true) stack'i kilitler, sayfa donar.

Benzetme Konsol, yazım denetimi gibi: hatayı bulur ve kırmızıyla gösterir. Okumayı öğren.

KOD ÇALIŞINCA

Kod yazılınca neler olur?

1- Okur

Motor kodu ayrıştırır, hata varsa durur.

2- Derler

JIT ile makine koduna çevirir.

3- Çalıştırır

Satırlar stack'te yürür.

4- Bekler

Event loop yeni iş gelmesini kollar.

Olmazsa olmaz: Stack'i uzun işle bloklama. Hataları yakala (try/catch, .catch). await'i unutma. Her zaman === kullan.

Benzetme Tarif okuyup (parse), mutfakta pişirip (çalıştır), garsonla servis etmek (event loop) gibi.

ÖZET

Bugün ne öğrendik?

JavaScript
Dinamik, tek iş parçacıklı dil.

Call Stack
Çalışan işlerin yığını (LIFO).

Web API
Bekleyen işleri arka planda yürütür.

Event Loop
Stack boşsa kuyruktan iş taşır.

Microtask ve Task
Promise'ler setTimeout'tan önce çalışır.

async / await
Asenkronu senkron gibi yazdırır.

Benzetme Event loop bir orkestra şefi gibi: kimin ne zaman çalacağını o belirler.

Dinlediğiniz için teşekkürler

Sorularınızı bekliyoruz.

Ayşe Nurcihan Akabaş · Altay